

数值计算方法 第三次上机作业

自动化（控制）

1 问题描述

考虑如下线性方程组：

$$\begin{cases} 0.4096x_1 + 0.1234x_2 + 0.3678x_3 + 0.2943x_4 = 0.4043 \\ 0.2246x_1 + 0.3872x_2 + 0.4015x_3 + 0.1129x_4 = 0.1550 \\ 0.3645x_1 + 0.1920x_2 + 0.3781x_3 + 0.0643x_4 = 0.4240 \\ 0.1784x_1 + 0.4002x_2 + 0.2786x_3 + 0.3927x_4 = -0.2557 \end{cases} \quad (1)$$

要求以 4 位有效数字的精度编程计算：

- (a) 用原始高斯消去法解上述方程组；
- (b) 用列主元消去法解上述方程组并与 (a) 比较结果。

在 MATLAB 中用函数 `round(X,N,'significant')` 限制有效位。

2 问题分析

本题要求在有限精度（4 位有效数字）下求解一个 4×4 线性方程组，并对比原始高斯消去法与列主元消去法在相同精度限制下的求解结果。

两种方法的本质区别在于主元的选取策略：原始高斯消去法按行序直接选取当前对角元作为主元，当某步主元 A_{kk} 较小时，消元因子 $\text{factor} = A_{ik}/A_{kk}$ 可能远大于 1，进而在低精度截断下产生较大的舍入误差并逐步积累；列主元法在每步消元前从当前列选取绝对值最大的元素作为主元并交换行，使消元因子始终满足 $|\text{factor}| \leq 1$ ，从而有效抑制舍入误差的传播放大。

本题对每一步四则运算的结果均立即截断至 4 位有效数字，具体包括：消元因子的计算、矩阵元素的更新、回代过程中的累减以及最终的除法。截断操作通过以下函数实现：

`round(x, 4, 'significant')`

这种“每步截断”的策略能够真实模拟低精度计算环境中舍入误差的逐步积累过程，与仅在最终输出时截断有本质区别——前者会将每步引入的误差传入后续运算，后者则掩盖了中间误差的传播。

3 原始高斯消去法

3.1 算法分析

原始高斯消去法通过逐列消元将增广矩阵 $[A|b]$ 化为上三角形式，再经回代求解。第 k 步消元对每行 $i > k$ 按如下公式更新：

$$\text{factor} = \frac{A_{ik}}{A_{kk}}, \quad A_{ij} \leftarrow A_{ij} - \text{factor} \cdot A_{kj} \quad (j = k, \dots, n+1) \quad (2)$$

消元完成后，回代阶段从最后一行开始逐行求解：

$$x_i = \frac{b_i - \sum_{j=i+1}^n A_{ij} x_j}{A_{ii}} \quad (3)$$

本题在每次四则运算后立即截断至 4 位有效数字，以模拟低精度计算环境中舍入误差的真实积累。

3.2 MATLAB 程序

核心函数 `gauss_4sig` 实现如下（完整代码见附录）：

```
function x = gauss_4sig(A, b)
    n = size(A, 1);
    Ab = [A, b]; % 构建增广矩阵
    % 消元阶段
    for k = 1:n-1
        for i = k+1:n
            factor = round(Ab(i,k)/Ab(k,k), 4, 'significant');
            for j = k:n+1
                Ab(i,j) = round(Ab(i,j) - factor*Ab(k,j), 4, 'significant');
            end
        end
    end
    % 回代阶段
    x = zeros(n, 1);
    x(n) = round(Ab(n,n+1)/Ab(n,n), 4, 'significant');
    for i = n-1:-1:1
        s = Ab(i, n+1);
        for j = i+1:n
            s = round(s - Ab(i,j)*x(j), 4, 'significant');
        end
        x(i) = round(s/Ab(i,i), 4, 'significant');
    end
end
```

3.3 运行结果及分析

原始高斯消去法结果：

x1 = -0.2007

x1 = -1.6800

x1 = 2.2440

x1 = -0.4468

原始高斯消去法在 4 位有效数字精度下给出了上述数值解。对该方法的误差积累机制分析如下：

第一步消元：以 $A_{11} = 0.4096$ 为主元，消元因子最大约为 $0.3645/0.4096 \approx 0.890$ ，小于 1，第一步误差放大尚不显著。

后续消元步骤：经第一步消元后矩阵结构发生改变，缩减子矩阵的对角元 $A_{22}^{(2)}$, $A_{33}^{(3)}$ 等均由原始元素与消元结果相减得到。在 4 位精度截断下，这些元素本身已含有舍入误差；若某步对角元恰好因截断而偏小，则该步消元因子 $|\text{factor}| = |A_{ik}/A_{kk}|$ 可能超过 1，使当前步的截断误差被等比放大后写入矩阵，参与下一步运算，形成误差的链式传播。

回代阶段：误差在回代的累减过程中进一步叠加。每个 x_i 的计算依赖已求出的 $x_{i+1}, \dots, x_n$ ，后者携带的截断误差通过 $A_{ij} \cdot x_j$ 传递至 x_i ，使得靠前的分量（如 x_1, x_2 ）往往误差更大。

综上，原始高斯消去法在低精度计算中，误差通过“消元放大—矩阵写回—再次参与消元”的循环机制持续积累，导致最终结果的可靠性较差。

4 列主元消去法

4.1 算法分析

列主元消去法在第 k 步消元开始前，在 $A_{kk}, A_{k+1,k}, \dots, A_{nk}$ 中寻找绝对值最大的元素所在行 p ，若 $p \neq k$ 则将增广矩阵第 p 行与第 k 行整体交换，再执行与原始高斯消去法相同的消元步骤。

行交换使得每步消元因子满足 $|\text{factor}| \leq 1$ ，从而限制了舍入误差在消元过程中的放大，在低精度计算环境下具有更好的数值稳定性。

4.2 MATLAB 程序

函数 `gauss_pivot_4sig` 在消元循环开头插入选主元与换行逻辑（完整代码见附录）：

```
function x = gauss_pivot_4sig(A, b)
    n = size(A, 1);
    Ab = [A, b];
    for k = 1:n-1
        % 列主元：选当前列绝对值最大元素所在行
        [~, idx] = max(abs(Ab(k:n, k)));
```

```

    p = idx + k - 1;
    if p ~= k
        Ab([k, p], :) = Ab([p, k], :); % 整行交换
    end
    % 消元（同原始高斯消去法）
    for i = k+1:n
        factor = round(Ab(i,k)/Ab(k,k), 4, 'significant');
        for j = k:n+1
            Ab(i,j) = round(Ab(i,j) - factor*Ab(k,j), 4, 'significant');
        end
    end
end
% 回代（同原始高斯消去法）
x = zeros(n, 1);
x(n) = round(Ab(n,n+1)/Ab(n,n), 4, 'significant');
for i = n-1:-1:1
    s = Ab(i, n+1);
    for j = i+1:n
        s = round(s - Ab(i,j)*x(j), 4, 'significant');
    end
    x(i) = round(s/Ab(i,i), 4, 'significant');
end
end
end

```

4.3 运行结果及分析

列主元消去法结果：

```

-0.1817
-1.6630
 2.2170
-0.4467

```

列主元法在第一步消元前，从第一列 $[0.4096, 0.2246, 0.3645, 0.1784]^T$ 中选出绝对值最大的 0.4096（恰好已在第一行，无需交换）；第二步则从缩减子矩阵的第二列中重新选取最大元素换至对角位置，依此类推。

行交换确保每步 $|\text{factor}| \leq 1$ ，即每步消元中截断误差**不被放大**：设当前步引入的截断误差量级为 δ ，则写回矩阵的误差量级至多也是 δ ，而不会因 $|\text{factor}| > 1$ 变成 $k\delta$ ($k > 1$)。这种“误差不增长”的性质在每一步消元中重复保证，经过全部消元与回代后，累积误差始终处于可控范围，最终结果也更为可靠。

与原始高斯消去法的结果相比， x_3 分量的差值达 0.0270，约占 $|x_3| \approx 2.2$ 的 1.2%。对于仅有 4 位有效数字的计算精度而言，这一差距是显著的，充分说明主元选取策略对低精度数值计算结果有实质性影响。

5 结果比较与分析

5.1 数值结果对比

两种方法的计算结果及差值如下表所示：

方法	x_1	x_2	x_3	x_4
原始高斯消去法	-0.2007	-1.6800	2.2440	-0.4468
列主元消去法	-0.1817	-1.6630	2.2170	-0.4467
差值 ($x_2 - x_1$)	0.0190	0.0170	-0.0270	0.0001

表 1: 两种方法的计算结果对比

5.2 误差来源分析

在足够高的精度下，两种方法理论上应给出相同结果。而在仅有 4 位有效数字的限制下，两者出现了明显差异： x_1 相差 0.0190， x_2 相差 0.0170， x_3 相差高达 0.0270，唯有 x_4 几乎吻合（差值仅 0.0001）。这种“有的分量差异大、有的差异小”的不均匀分布，正是舍入误差在消元和回代过程中非均匀积累的典型特征——不同分量对中间步骤误差的敏感程度不同，导致最终误差大小各异。

从误差传播的角度看，原始高斯消去法的问题在于：若第 k 步的主元 $|A_{kk}|$ 相对当前列其他元素偏小，则消元因子 $|\text{factor}| = |A_{ik}/A_{kk}|$ 可能超过 1，每次截断引入的绝对误差被放大后写入矩阵，参与后续计算时进一步与新的截断误差叠加，最终在回代阶段将各步积累的误差“汇总”至解向量。列主元法通过行交换保证 $|\text{factor}| \leq 1$ ，截断误差在每步乘法后不被放大，从根本上切断了误差链式放大的路径，数值稳定性显著更优。

5.3 方法综合对比

对比维度	原始高斯消去法	列主元消去法
主元选取	按行序，无控制	当前列绝对值最大元素
消元因子范围	无限制，可能 $\gg 1$	$ \text{factor} \leq 1$
低精度稳定性	较差，误差易积累	较好，误差受控
实现复杂度	简单	略增（多一步行交换）

表 2: 两种方法综合对比

6 心得体会

通过本次上机作业，我对“算法正确”与“算法稳定”之间的区别有了直观认识。原始高斯消去法和列主元法在数学上等价，在无限精度下给出完全相同的解；然而一旦

精度受限，两者的结果便出现了最大达 0.0270 的差异，这完全源于算法在有限精度下的数值稳定性之别。

更深刻的体会是：截断误差的危害不在于单次运算有多大，而在于它能否在后续运算中被放大并传播。列主元法以一次行交换的极小代价，将消元因子限制在 $[-1, 1]$ 内，从根源上阻断了误差的链式放大，控制中间量的大小以抑制误差传播的思路在数值计算的许多场合都有类似的体现，值得在今后的学习中加以关注和运用。

7 附：完整 MATLAB 程序及运行结果

7.1 完整 MATLAB 程序

```
clear; clc;

%% 定义矩阵 A 和向量 b
A = [0.4096, 0.1234, 0.3678, 0.2943;
      0.2246, 0.3872, 0.4015, 0.1129;
      0.3645, 0.1920, 0.3781, 0.0643;
      0.1784, 0.4002, 0.2786, 0.3927];
b = [0.4043; 0.1550; 0.4240; -0.2557];

%% (a) 原始高斯消去法
x1 = gauss_4sig(A, b);
fprintf('原始高斯消去法结果:\n');
fprintf(' x1 = %.4f\n', x1);

%% (b) 列主元消去法
x2 = gauss_pivot_4sig(A, b);
fprintf('列主元消去法结果:\n');
disp(x2);

%% 比较两种方法
fprintf('差值 (x2 - x1):\n');
disp(x2 - x1);

%% ----- 局部函数 -----
function x = gauss_4sig(A, b)
    n = size(A, 1);
    Ab = [A, b];
    for k = 1:n-1
        for i = k+1:n
            factor = round(Ab(i,k)/Ab(k,k), 4, 'significant');
            for j = k:n+1
                Ab(i,j) = round(Ab(i,j) - factor*Ab(k,j), 4, 'significant');
```

```

        end
    end
end
x = zeros(n, 1);
x(n) = round(Ab(n,n+1)/Ab(n,n), 4, 'significant');
for i = n-1:-1:1
    s = Ab(i, n+1);
    for j = i+1:n
        s = round(s - Ab(i,j)*x(j), 4, 'significant');
    end
    x(i) = round(s/Ab(i,i), 4, 'significant');
end
end

function x = gauss_pivot_4sig(A, b)
    n = size(A, 1);
    Ab = [A, b];
    for k = 1:n-1
        [~, idx] = max(abs(Ab(k:n, k)));
        p = idx + k - 1;
        if p ~= k
            Ab([k, p], :) = Ab([p, k], :);
        end
        for i = k+1:n
            factor = round(Ab(i,k)/Ab(k,k), 4, 'significant');
            for j = k+1:n
                Ab(i,j) = round(Ab(i,j) - factor*Ab(k,j), 4, 'significant');
            end
        end
    end
    x = zeros(n, 1);
    x(n) = round(Ab(n,n+1)/Ab(n,n), 4, 'significant');
    for i = n-1:-1:1
        s = Ab(i, n+1);
        for j = i+1:n
            s = round(s - Ab(i,j)*x(j), 4, 'significant');
        end
        x(i) = round(s/Ab(i,i), 4, 'significant');
    end
end
end

```

7.2 完整运行结果

原始高斯消去法结果：

```
x1 = -0.2007
```

```
x1 = -1.6800
```

```
x1 = 2.2440
```

```
x1 = -0.4468
```

列主元消去法结果：

```
-0.1817
```

```
-1.6630
```

```
2.2170
```

```
-0.4467
```

差值 (x2 - x1):

```
0.0190
```

```
0.0170
```

```
-0.0270
```

```
0.0001
```